

ESP32 web dijagnostički terminal za Micromouse robota preko UART-a

Programiranje mikrokontrolerskih sustava

David Mihić
Mehatronika i robotika

Ljetni semestar 2026.

Mentor: doc. dr. sc. Matija Krznar

1. Cilj projekta

Cilj projekta je dijagnostički modul (dashboard) na ESP32 mikrokontroleru koji služi kao web sučelje između operatera i Micromouse robota. Robot je izgrađen oko STM32G474 mikrokontrolera i preko UART-a kontinuirano šalje telemetriju. ESP32 te podatke prima, podiže vlastitu Wi-Fi pristupnu točku i poslužuje web stranicu na kojoj operater vidi stanje robota i može mu slati naredbe. Ne treba dodatni alat ni kabel prema računalu: operater se spoji na Wi-Fi mrežu robota i otvori stranicu u pregledniku.

Konkretno:

- Sustav prikazuje stanje robota uživo: IR senzore, žiroskop, odometriju, brzine kotača, pozu, stanje tipki, DIP prekidača i Neopixel LED diode.
- Rješava se problem otklanjanja grešaka na robotu bez žičanog spoja na računalu i bez prekida rada robota.
- ESP32 je posrednik (bridge): pretvara UART vezu prema robotu u Wi-Fi i HTTP koji preglednik razumije.
- Korisnik može gledati telemetriju i slati naredbe (npr. reset odometrije) kroz serijsku konzolu na stranici.

2. Funkcionalni zahtjevi

- Uspostaviti UART komunikaciju između ESP32 i STM32 robota.
- Konfigurirati UART parametre: 115200 bit/s, 8 podatkovnih bitova, bez pariteta, 1 stop bit (8N1).
- Primati poruke s UART-a, sastavljati ih u cijele linije i činiti ih dostupnima web sloju.
- Omogućiti slanje tekstualnih naredbi s web stranice prema robotu preko UART-a.
- Prikazati status komunikacije (živa veza / nema podataka).
- Parsirati dogovoreni format telemetrije (JSON linija) i prikazati polja u odvojenim karticama.
- Validirati format poruka i odbaciti prazne ili neispravne linije.
- Važne događaje ispisivati u ESP-IDF serijski terminal radi debugiranja.

3. Korišteni hardver

Element	Primjer	Namjena
Mikrokontroler (dashboard)	ESP32 / ESP32-S3	Wi-Fi AP, web server, UART most
Robot	STM32G474CET6	izvor telemetrije, prima naredbe
UART konektor	B4B-PH na robotu	nosi UART_TX, UART_RX, +3V3, GND
Napajanje	USB / vlastiti regulatori	odvojeno za ESP32 i robota

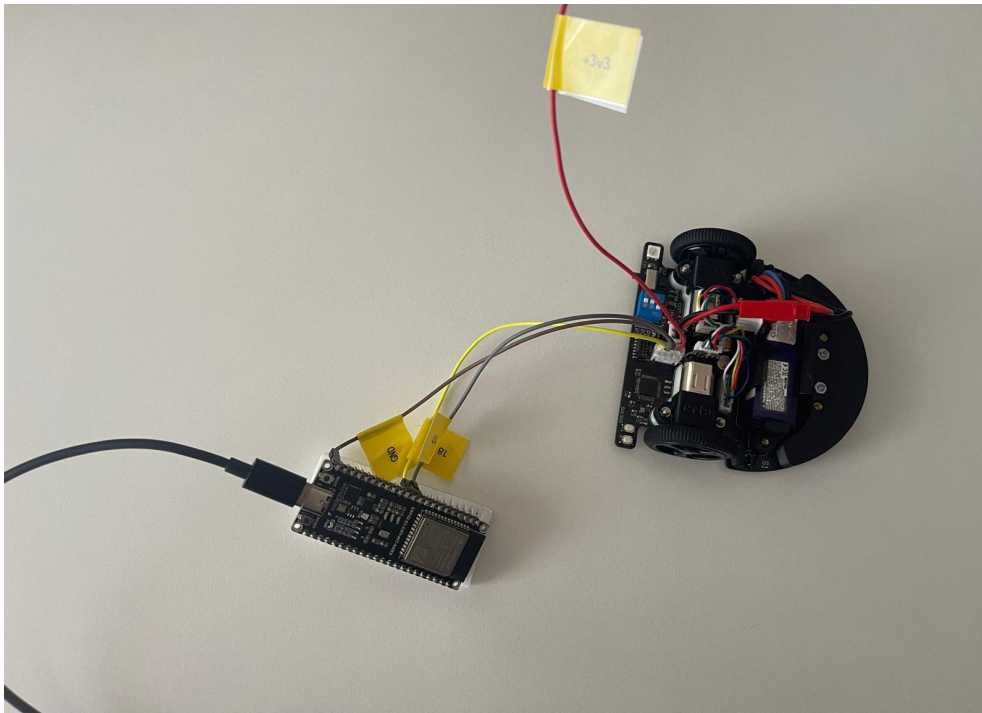
UART linije na robotu su 3.3 V logičke razine (STM32 radi na 3.3 V), pa nije potreban level shifter. Napajanja ESP32 i robota ne spajaju se međusobno; dijeli se samo masa.

4. Shema spajanja

Spoj je UART, izveden križno: TX jedne strane ide na RX druge strane. Na robotu je to USART2 (PA2 = TX, PA3 = RX), izveden na B4B-PH konektor. Na ESP32 strani koristi se UART1 (TX = GPIO17, RX = GPIO18).

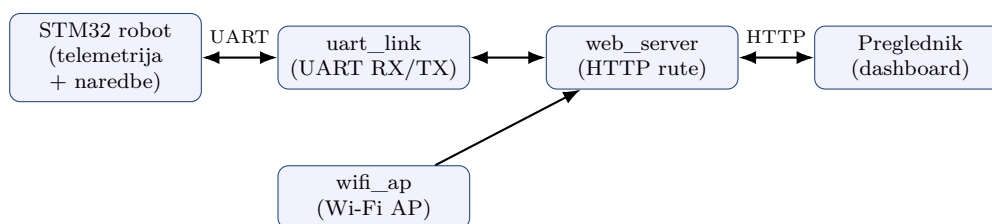
Robot (STM32)	Smjer	ESP32
UART_TX (PA2)	→	RX (GPIO18)
UART_RX (PA3)	←	TX (GPIO17)
GND	—	GND

Obavezno je spojiti zajedničku masu. Napajanje (+3V3) s konektora ne spaja se na ESP32 ako oba uređaja imaju vlastito napajanje.



Slika 1: Spoj ESP32 i robota preko UART-a (TX, RX, GND).

5. Blok dijagram sustava



Blokovi:

- **STM32 robot** – šalje JSON telemetriju i izvršava primljene naredbe.
- **uart_link** – ESP-IDF komponenta; čita bajtove s UART-a, sastavlja linije i čuva zadnju primljenu liniju; šalje naredbe robotu.
- **wifi_ap** – podiže Wi-Fi pristupnu točku (SSID Micromouse, IP 192.168.4.1).
- **web_server** – HTTP server koji posluhuje stranicu i posreduje telemetriju i naredbe.
- **Preglednik** – prikazuje podatke i šalje naredbe; periodički dohvaća telemetriju.

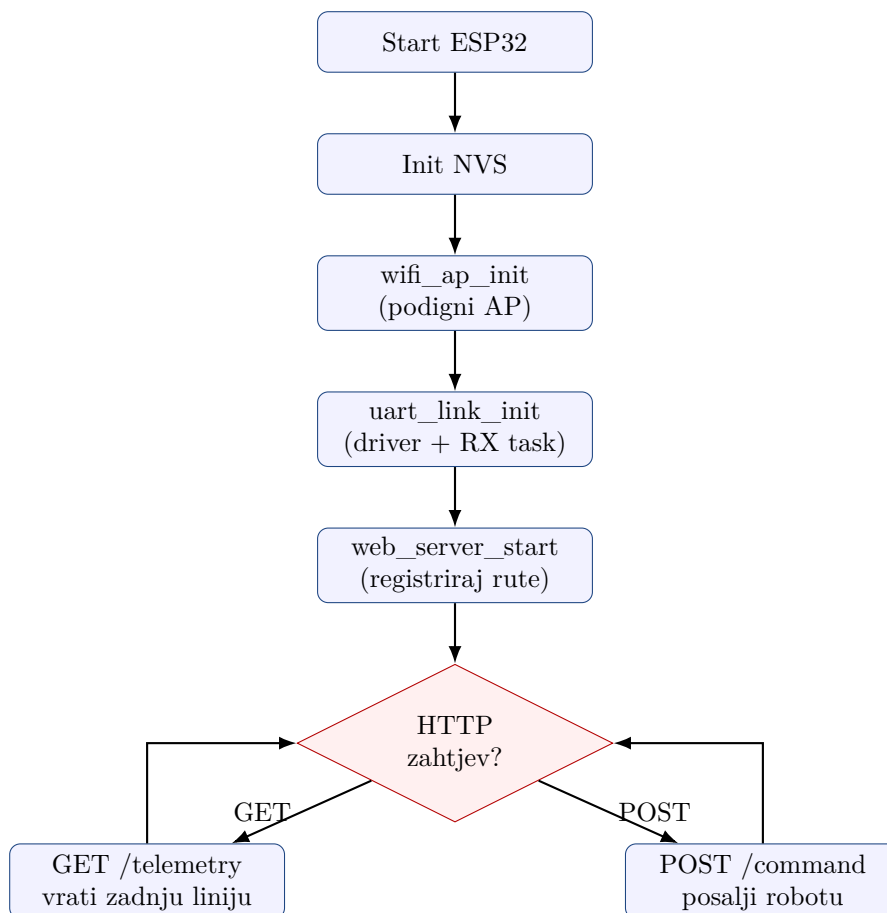
6. Programska struktura

Projekt je ESP-IDF aplikacija podijeljena na komponente. Svaka komponenta ima jednu odgovornost, što olakšava testiranje i izmjene neovisno o ostatku sustava.

Cjelina	Datoteka	Zadatak
Glavni program	main/main.c	inicijalizacija NVS-a i redom pokretanje AP-a, UART-a i web servera
Wi-Fi AP	wifi_ap.c/h	konfiguracija i pokretanje pristupne točke
UART most	uart_link.c/h	UART driver, RX task, slanje naredbi
Web server	web_server.c/h	HTTP server, ugrađene statičke datoteke, rute
Frontend	index.html, script.js, style.css	prikaz telemetrije i serijska konzola u pregledniku

Statičke datoteke (`index.html`, `script.js`, `style.css`) ugrađene su u firmware preko `MBED_TXTFILES` u `CMakeLists.txt` web servera, pa nije potreban datotečni sustav na flashu.

7. Flow dijagram



Najvažniji koraci:

- NVS se inicijalizira jer ga Wi-Fi stog zahtijeva (kalibracije, MAC i sl.).
- RX task u pozadini neprestano čita UART i ažurira zadnju primljenu liniju.
- Web server je vođen zahtjevima: na `GET /telemetry` vraća zadnju liniju, na `POST /command` prosljeđuje naredbu robotu.

8. Pseudokod

Pseudokod 1: Inicijalizacija (app_main)

```
app_main:
  init NVS (erase + retry on error)
  wifi_ap_init()
  uart_link_init()
  web_server_start()
  log("Dashboard ready, http://192.168.4.1")
```

Pseudokod 2: RX task (sastavljanje linija s UART-a)

```
rx_task:
  loop forever:
    n = uart_read_bytes(chunk, timeout)
    for each byte c in chunk:
      if c is newline:
        if line not empty and line starts with '{':
          commit_line(line) # spremi pod mutexom
          reset line
        else if line not full:
          append c to line
        else:
          drop line (overflow)
```

Pseudokod 3: Preglednik (poll petlja)

```
every 50 ms:
  try:
    d = GET /telemetry # JSON
    render(d) # popuni kartice
    status = "live"
  catch:
    status = "nema podataka"

on console Enter:
  cmd = input (lowercase, trimmed)
  if cmd == "clear": clear console
  else: POST /command body=cmd
```

9. Web sučelje ili komunikacija

Korisnik se spoji na Wi-Fi mrežu Micromouse (lozinka micromouse, WPA2) i otvori <http://192.168.4.1>. Stranica je podijeljena na kartice (IR senzori, žiroskop, poza, odometrija, brzine, gibanje, tipke, DIP, Neopixel) i serijsku konzolu.

HTTP rute (na ESP32):

Ruta	Metoda	Opis
/	GET	početna web stranica (HTML)
/script.js	GET	klijentska logika
/style.css	GET	stilovi
/telemetry	GET	zadnja primljena telemetrija (JSON)
/command	POST	tijelo zahtjeva je naredba za robota

UART protokol. Tekstualni je i jednostavan za debugiranje. Svaka poruka završava znakom za novi red. Telemetrija je jedna JSON linija; ESP prihvaća samo linije koje počinju znakom { (osnovna validacija), a sve ostalo (debug ispisi robota) ignorira. Naredbe prema robotu su kratke tekstualne riječi.

Podržane naredbe:

Naredba	Učinak
<code>reset_odom</code>	resetira odometriju, enkodere i kut <code>yaw</code> na robotu
<code>cycle_led</code>	uključuje/isključuje cikličko mijenjanje boje Neopixela
<code>clear</code>	briše konzolu (obrađuje se samo u pregledniku)

Primjer telemetrije (JSON odgovor na `/telemetry`):

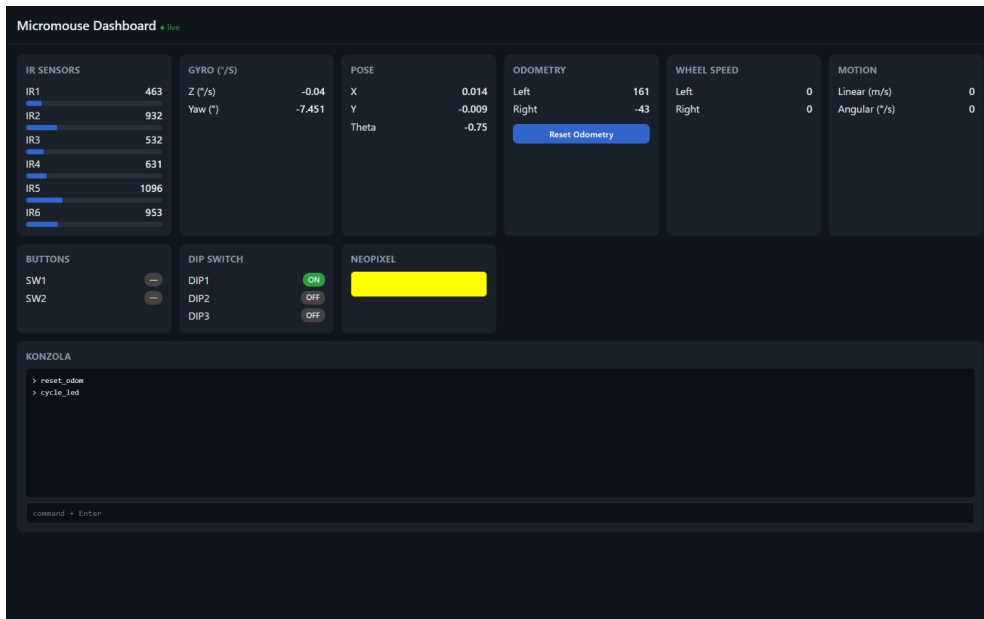
```
{
  "ir": [120, 95, 30, 28, 88, 110],
  "gyro": { "Z": -1.25 },
  "Yaw": 12.430,
  "odom": { "Left": 1024, "Right": 1019 },
  "speed": { "Left": 3.14, "Right": 3.10 },
  "pose": { "X": 0.182, "Y": 0.004, "Theta": 0.217 },
  "v": 0.420, "w": 0.030,
  "btn": { "SW1": 0, "SW2": 1 },
  "dip": 5,
  "led": { "r": 0, "g": 100, "b": 0 }
}
```

Polja: `ir` su 6 IR prijemnika (ADC očitavanja), `gyro.Z` kutna brzina po Z osi, `Yaw` integrirani kut, `odom` pozicija enkodera u tickovima, `speed` brzine kotača, `pose` poza robota, `v/w` linearna i kutna brzina, `btn` stanje tipki (1 = pritisnuto), `dip` bitmaska 3 DIP prekidača, `led` trenutna RGB boja Neopixela. Polje `dip` se na klijentu rastavlja po bitovima (`dip » i & 1`).

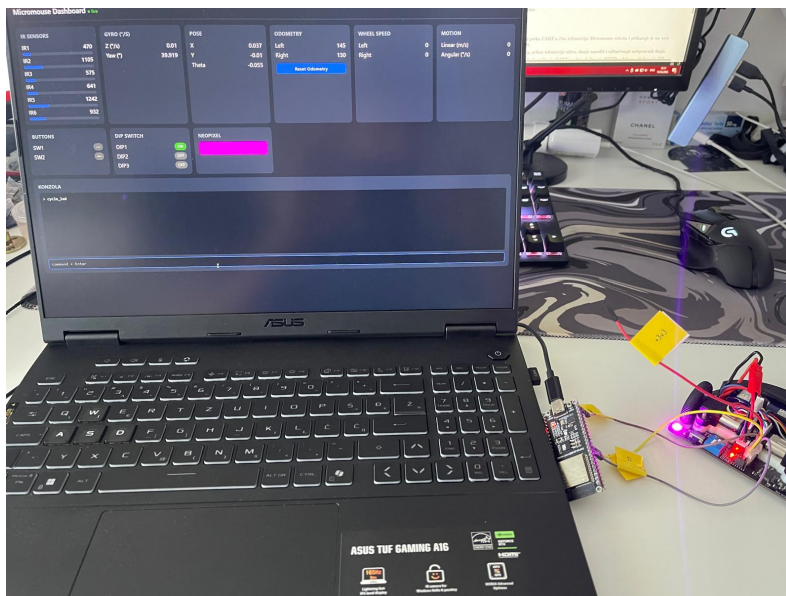
10. Testiranje

Test	Očekivani rezultat	Status
Pokretanje ESP32	U logu se vidi pokrenut AP i HTTP server	OK
Spoj na Wi-Fi i otvaranje stranice	Stranica se učitava na <code>192.168.4.1</code>	OK
Prijem telemetrije	Kartice se ažuriraju, status pokazuje 'live'	OK
Odspajanje UART-a	Kartice prazne, status 'nema podataka'	OK
Slanje <code>reset_odom</code>	Odometrija i <code>yaw</code> na robotu se postave na 0	OK
Slanje <code>cycle_led</code>	Neopixel ledica na robotu se pali i ciklički mijenja boje	OK
Slanje <code>clear</code>	Konzola u pregledniku se briše	OK
Neispravna linija na UART-u	Linija bez { se ignorira	OK

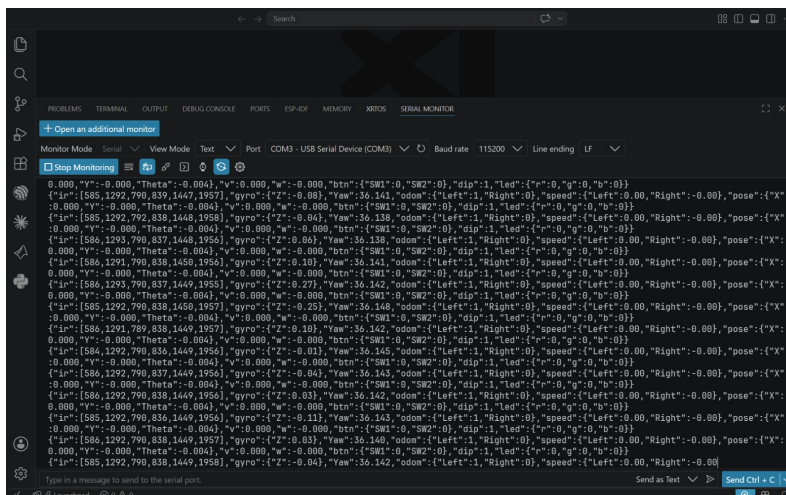
Dokazi rada prikazani su na slikama [2-4](#).



Slika 2: Web sučelje s aktivnom telemetrijom.



Slika 3: Laptop s pokrenutim dashboardom i robot pokraj njega.



Slika 4: Ispis na serijski monitor.

11. Zaključak

- Napravljen je ESP32 dashboard koji preko UART-a čita telemetriju Micromouse robota i prikazuje je na web stranici, uz mogućnost slanja naredbi.
- Uspješno su testirani: podizanje AP-a, prikaz telemetrije uživo, slanje naredbi i odbacivanje neispravnih linija.
- Glavni izazov bio je odvajanje pozadinskog čitanja UART-a od posluživanja HTTP zahtjeva; riješeno je RX taskom i dijeljenjem zadnje linije pod mutexom.
- Moguća poboljšanja: detekcija gubitka veze po isteku vremena (npr. 5 s bez nove linije, a ne samo po HTTP grešci), pohrana postavki veze u NVS, te prelazak s pollinga na WebSocket radi manjeg opterećenja.